



UNIVERSIDAD NACIONAL AUTÓNOMA DE  
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



**PREVIO N° 08**

**NOMBRE COMPLETO:** Del Toro Ortiz Juan Pablo Yasbeth

**N° de Cuenta:** 317031814

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 7

**SEMESTRE** 2026-2

**FECHA DE ENTREGA LÍMITE:** 16/04/2026

**CALIFICACIÓN:** \_\_\_\_\_

## Previo 7: Iluminación:

### 1.- ¿Cómo funciona la técnica de Cocinado de luces? ( No explicar usando Motores Gráficos como Unity, Unreal ni Software de Modelado)

**R=** La técnica de "cocinado de luces" (*light baking*) es un proceso que precalcula la iluminación de una escena 3D y la guarda en unas texturas llamadas mapas de luz (lightmaps). Luego, en tiempo real, el juego aplica estos mapas de luz sobre los objetos, evitando tener que recalcular toda la iluminación en cada frame

Esto permite lograr efectos visuales de alta calidad como el rebote de color (*color bouncing*), sombras suaves y una iluminación global mucho más realista, al mismo tiempo que reduce la carga de trabajo de la GPU y la CPU. La desventaja principal es que es un proceso estático: las luces no pueden moverse ni cambiar, y el proceso de cocinado puede consumir mucho tiempo (*Light Baking*, n.d.)

**R=** El light baking, también conocido como lightmapping, es un proceso de generación de mapas de texturas que almacenan la información de iluminación de un modelo 3D o una escena. Estos mapas pueden incluir iluminación directa e indirecta, sombras, reflejos, oclusión ambiental e iluminación global. A continuación, los mapas se aplican a la superficie del modelo o de la escena como una capa adicional de textura, creando la ilusión de una iluminación dinámica sin necesidad de cálculos complejos. Se puede hacer un horneado ligero para modelos individuales o para escenas completas, según el nivel de detalle y complejidad requerido. (*¿Cómo Puedes Usar El Horneado Ligero Para Mejorar La Iluminación Y El Renderizado De Tu Juego?*, 2023)

### 2.- ¿Cómo funciona la iluminación diferida en OpenGL y shader?

**R=** Esta técnica divide el renderizado en dos fases principales mediante el uso de un G-Buffer (Geometry Buffer). En la primera fase (Geometry Pass), se renderizan las propiedades de la geometría (posición, normales, albedo y especularidad) en múltiples texturas simultáneas mediante MRT (Multiple Render Targets).

En la segunda fase (Lighting Pass), el shader calcula la iluminación de cada píxel utilizando únicamente la información almacenada en las texturas del G-Buffer. Esto permite manejar cientos de luces con un costo computacional que depende de la resolución de la pantalla y no de la complejidad de la geometría (*LearnOpenGL - Deferred Shading*, n.d.)

### 3.- ¿Para qué se utiliza una variable del tipo deltaTime en OpenGL y animación?

**R=** La variable deltaTime se utiliza en OpenGL y animación para hacer que el movimiento y las animaciones sean independientes de la velocidad de fotogramas (FPS). Representa el tiempo transcurrido entre el frame actual y el frame anterior. Al multiplicar la velocidad de un objeto por este deltaTime, se asegura que recorra la misma distancia en un segundo, sin importar si el juego corre a 30 o a 200 FPS. Esto evita que el movimiento se sienta demasiado rápido en monitores de alta tasa de refresco o demasiado lento en equipos con bajos FPS (Byc, 2017)

**R=** La variable deltaTime representa el tiempo transcurrido (usualmente en segundos o milisegundos) entre el cuadro (frame) actual y el anterior. Se utiliza para garantizar la independencia de la tasa de cuadros (frame rate independence). Al multiplicar los cambios de magnitud física —como la velocidad o rotación— por el deltaTime, se asegura que el movimiento en la simulación sea constante respecto al tiempo real, evitando que los objetos se muevan más rápido en computadoras con hardware potente o más lento en equipos con menor rendimiento (*LearnOpenGL - Camera*, n.d.)

## Conclusión

Al realizar este cuestionario de investigación, pude comprender que existen diferentes estrategias para manejar la iluminación y el movimiento en gráficos 3D. Aprendí que el "cocinado de luces" (light baking) precalcula la iluminación y la almacena en texturas llamadas lightmaps, lo que ahorra recursos en tiempo real porque la luz ya no tiene que calcularse en cada frame. Sin embargo, su principal limitación es que las luces no pueden moverse ni cambiar una vez que han sido cocinadas.

También entendí que la iluminación diferida en OpenGL soluciona el problema de manejar muchas luces al separar el proceso en dos fases: primero guarda la información de la geometría en un G-Buffer y luego calcula la iluminación por cada píxel. Esto permite tener cientos de luces de manera eficiente, ya que el costo depende del tamaño de la pantalla y no de la cantidad de objetos.

Por último, comprendí que la variable deltaTime es clave para que los movimientos en una escena 3D no dependan de la velocidad de renderizado. Al multiplicar la velocidad de un objeto por deltaTime, se logra que recorra la misma distancia en un segundo sin importar si el renderizado corre a 30 o a 200 fotogramas por segundo, manteniendo así la consistencia del movimiento en cualquier equipo.

## Bibliografía

- Light baking. (n.d.). [https://help.autodesk.com/view/Stingray/ENU/?guid=\\_\\_stingray\\_help\\_lighting\\_rendering\\_lighting\\_light\\_baking\\_html](https://help.autodesk.com/view/Stingray/ENU/?guid=__stingray_help_lighting_rendering_lighting_light_baking_html)
- ¿Cómo puedes usar el horneado ligero para mejorar la iluminación y el renderizado de tu juego? (2023, September 29). [www.linkedin.com](https://es.linkedin.com/advice/1/how-can-you-use-light-baking-improve-your-games-lighting?lang=es).  
<https://es.linkedin.com/advice/1/how-can-you-use-light-baking-improve-your-games-lighting?lang=es>
- LearnOpenGL - Deferred Shading. (n.d.). <https://learnopengl.com/Advanced-Lighting/Deferred-Shading>
- Byc. (2017, October 25). ¿Por qué necesitamos utilizar Delta Time? – Parallelcube. <https://www.parallelcube.com/es/2017/10/25/por-que-necesitamos-utilizar-delta-time/>
- *LearnOpenGL - Camera*. (n.d.). <https://learnopengl.com/Getting-started/Camera>